

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Organización de Lenguajes y Compiladores 1

MANUAL DE USUARIO

Dionicio Dario Simon Bal

201905487

Guatemala 23 de Octubre del 2024

Descripción General

ComInterpreter es un intérprete sencillo capaz de traducir código y realizar una ejecución basada en JavaScript posee una interfaz gráfica en la cual nos permite ejecutar el código con botón,

El lenguaje mostrará en consola todas las sentencias impresas, también será capaz de indicar los errores sintácticos, léxicos y semánticos encontrados durante la ejecución indicando la posición en la cual se encuentra el problema

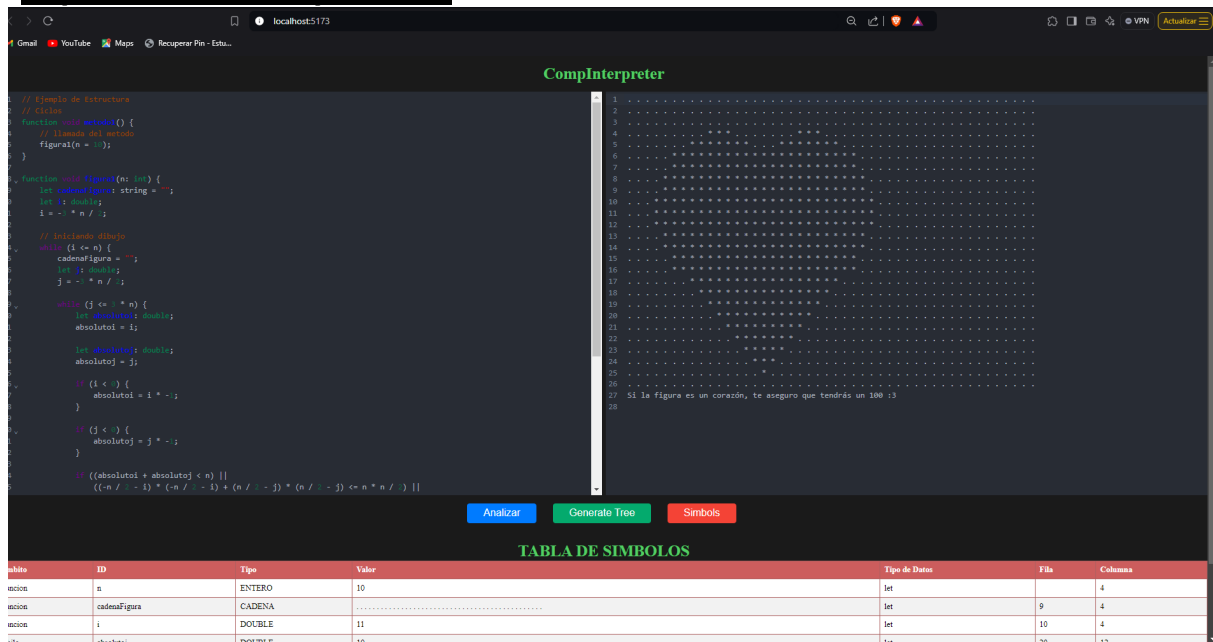
La aplicación nos permite guardar los archivos que escribamos, también nos permitirá visualizar el árbol AST generado por la entrada ingresada en el área de programación.

Requerimientos mínimos del sistema

Para el uso correcto de la aplicación es necesario los siguientes componentes y programas:

- Memoria RAM 2GB o superior
- Espacio disponible en disco de 500 MB
- Angular versión 11.2.9
- JavaScript 4.2.4 o superior
- NodeJS 6.14.7
- Graphviz versión 2.38.0 (20140413.2041)
- Sistema Operativo Windows 10 x64

Explicación de la Aplicación



CompInterpreter

```
// Ejemplo de Estructura
// Cálculo
function calcResultado() {
  // Inicialización de variables
  figura(n = 10);
}

function void figura(n: int) {
  let cadenaFigura: string = "";
  let i: double;
  let j: double;

  // Inicialización de variables
  calcFigura = "";
  let i: double;
  j = -1 * n / 2;

  while (j <= 1 * n) {
    let absolutoi: double;
    absolutoi = i;
    let absolutoj: double;
    absolutoj = j;

    if (i < 0) {
      absolutoi = i * -1;
    }

    if (j < 0) {
      absolutoj = j * -1;
    }

    if ((absolutoi + absolutoj < n) ||
        ((-n / 2 - i) * (-n / 2 - i) + (n / 2 - j) * (n / 2 - j) <= n * n / 2)) {
      cadenaFigura += " * ";
    } else {
      cadenaFigura += "   ";
    }

    i++;
    j++;
  }

  console.log(cadenaFigura);
}
```

TABLA DE SIMBOLOS

Nombre	ID	Tipo	Valor	Tipo de Datos	Fila	Columna
union	n	ENTERO	10	int		4
union	cadenaFigura	CADENA		let	9	4
union	i	DOUBLE	11	let	10	4
ila	absolutoi	DOUBLE	10	let	20	17

1. GENERATE TREE: este botón generara el árbol AST de la entrada ingresada
2. BOTON ANALIZAR: Este botón iniciara la ejecución del código indicado en la terminal

Explicación del Lenguaje

1. Tipos: la aplicación es capaz de manejar 5 tipos primitivos de datos entre los cuales tenemos char, int, double, boolean, string.
2. Declaración de variables: para declarar una variable "LET ID = TIPO EXPRESION" o con la primera declaración podemos asignarle a la variable un valor inicial, mientras que en la segunda forma obtendrá el valor por defecto de cada tipado.
3. Asignación de variables: para poder asignar una variable se utiliza la siguiente estructura "ID = EXPRESION" con dicha instrucción podemos asignar un nuevo valor a la variable;
4. Declaración de vector y listas: Para poder declarar un vector se utiliza la siguiente estructura "TIPO [] ID = NEW INT[EXP]" y la de una lista será "LIST <TIPO> ID = NEW LIST<TIPO>" existen diferentes variaciones de declaración, pudiendo realizarse la siguiente para el vector "TIPO [] ID = {EXP, EXP, ... EXP}" y para una lista "LIST <TIPO> ID = TOCHARARRAY(EXP)"
5. Asignación de vector y lista: la asignación de un vector es la siguiente "ID[EXP] = EXP" y la asignación de una lista es "ID[[EXP]] = EXP"
6. Agregar dato a lista; para agregar un nuevo elemento se utiliza la función add "ID.ADD(EXP)"
7. Ciclos
 - a. FOR: la estructura de un for es la siguiente
"FOR(DECLARACION/ASIGNACION; CONDICION; ACTUALIZACION){
INSTRUCCIONES}"
 - b. WHILE: la estructura de un while es la siguiente
"WHILE(CONDICION){INSTRUCCIONES}"
 - c. DO UNTIL: la estructura de un while es la siguiente "DO { INSTRUCCIONES}
UNTIL(CONDICION)"
8. Nativas: existen diferentes funciones nativas con la estructura "FUNCION(EXP)" en las cuales podemos mencionar, toString, length, round, truncate, toUpper, toLower
9. Funciones y métodos
 - a. funciones: los métodos tienen la siguiente estructura "FUNCTION TIPO ID (
PARAMETROS
) { INSTRUCCIONES }"
 - b. métodos: los métodos tienen la siguiente estructura "FUNCTION VOID ID (
PARAMETROS)
{ INSTRUCCIONES } "

10. EJECUTAR: es una forma de llamar una función y a la vez indicara cual es nuestra función principal a ejecutar, solo puede existir uno por terminal
“EJECUTAR ID (PARAMETROS) “

11. LLAMADAS: para llamar una función o un método se utiliza la siguiente estructura “ ID (PARAMETROS)”

Ejemplo de Estructura

```
// Ejemplo de Estructura
// Ciclos
function metodo1() {
    // llamada del metodo
    figura1(n = 10);
}
```

```
Function int figura1(n: int){
    let cadenaFigura: string = "";
    let i: double;
    i = -3 * n / 2;

    // iniciando dibujo
    while (i <= n) {
        cadenaFigura = "";
        let j: double;
        j = -3 * n / 2;

        while (j <= 3 * n) {
            let absolutoi: double;
            absolutoi = i;

            let absolutoj: double;
            absolutoj = j;

            if (i < 0) {
                absolutoi = i * -1;
            }

            if (j < 0) {
                absolutoj = j * -1;
            }

            if ((absolutoi + absolutoj < n) ||
                ((-n / 2 - i) * (-n / 2 - i) + (n / 2 - j) * (n / 2 - j) <= n * n / 2) ||
                ((-n / 2 - i) * (-n / 2 - i) + (-n / 2 - j) * (-n / 2 - j) <= n * n / 2)) {
                cadenaFigura = cadenaFigura + "* ";
            } else {
                cadenaFigura = cadenaFigura + ". ";
            }
            j = j + 1;
        }
        i = i + 1;
    }
}
```

```
Echo cadenaFigura;  
i = i + 1;  
}  
  
Echo "Si la figura es un corazón, te aseguro que tendrás un 100 :3";  
}  
  
// Ejecución del método  
EJECUTAR metodo1();
```

Salida Esperada

[illegible]

Si la figura es un corazon, estás en el buen camino :D

*/

Árbol AST

